DEPARTMENT OF AVIONICS ENGINEERING

Aviation and Aerospace University, Bangladesh

AAUB · “Space and Beyond”

// MODELING & SIMULATION SESSIONAL — LAB 01

LAB REPORT

Modeling and Simulation Sessional

```
course_code = 'AVE 4410';
```

Introduction to the MATLAB Environment

lab01_matlab_basics.m — Topics Covered

```
% 01  The MATLAB Environment and Basics
% 02  Scripts, Functions, and Documentation
% 03  Data Visualization and Plotting
% 04  Image Processing Basics
% 05  Introduction to Matrices and Operators
```

SUBMITTED BY

Md. Rezwan Zani

ID 24024021

Batch 04

Dept. of Avionics Engineering, AAUB

SUBMITTED TO

Md. Samin Rahmen

Assistant Professor

Dept. of Avionics Engineering, AAUB

Date .....

```
>> clc; clear; close all;  plot(t, sin(t)); grid on;  % AVE 4410 · Batch 04
```

LAB REPORT – 1

TOPIC – 1: The MATLAB Environment and Basics

CODE #1 [environment_basics.m]

```
% Reduce vertical spacing in the command window
format compact

% Generate 10 million random numbers (press Ctrl+C to stop)
rand(10000000, 1)

% Generate a single random number
rand

% Basic variable assignment and arithmetic
a = 1;
b = 2;
c = 3;
sum = a + b + c;

% Clear specific variables from the workspace
clear a
clear b

% Wipe out all variables
clear all

% Clear the command window
clc

% Save and load workspace variables
save
clear
load
```

OUTPUT

```
    0.9770
    0.1392
    0.0694
    0.7030
    0.8573
    0.6276
    0.5970
```

```
    0.3648
    0.4125
[Continues….]

ans =
    0.3039
```

CODE #2 [variable_naming_and_errors.m]

```
% MATLAB is case-sensitive
micky = 9;
micKy = 10;

% Variables can contain numbers, but cannot start with them
x45 = 10;
% 45x = 12 % Syntax error: invalid expression
% 5x = 10  % Syntax error

% Use a semicolon to execute a command without showing output
abs_zero_C = -273.15;
abs_zero_F = abs_zero_C * 9/5 + 32;
```

OUTPUT

Name	Value	Size	Class
abs_zero_C	-273.1500	1×1	double
abs_zero_F	-459.6700	1×1	double
micky	9	1×1	double
micKy	10	1×1	double
x45	10	1×1	double

CODE #3 [variable_swapping.m]

```
x = 42;
y = 38;

% Semantic error: this copies y into x, but doesn't swap them
x = y;
y = x;

% Correct way to swap values using a temporary variable
x = 42;
y = 38;
temp = x;
x = y;
y = temp;
```

OUTPUT

Name	Value	Size	Class
temp	42	1×1	double
x	38	1×1	double
y	42	1×1	double

CODE #4 [light_year_calculation.m]

```
% Calculate the number of kilometers in a lightyear
speed_kps = 300000; % Speed of light in km/s
year_sec = 365 * 24 * 60 * 60; % Seconds in a year
lightyear_km = year_sec * speed_kps;
```

OUTPUT

Name	Value	Size	Class
lightyear_km	9.4608e+12	1×1	double
speed_kps	300000	1×1	double
year_sec	31536000	1×1	double

TOPIC – 2: Scripts, Functions, and Documentation

CODE #1 [pretty_picture.m]

```
function pretty_picture(N)
%PRETTY_PICTURE Plot a growing cosine wave versus a growing sine
% PRETTY_PICTURE(N) plots with N points.
% PRETTY_PICTURE without input uses N = 500.
% Created 1985 by Cleve Moler and Jack Little

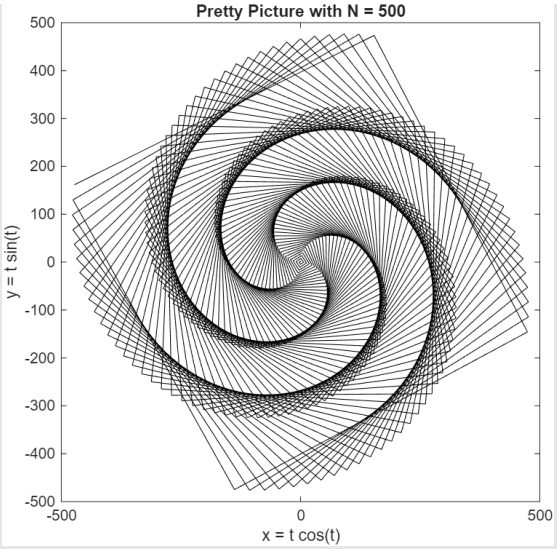
% If no input provided, use default N = 500
if nargin < 1
    N = 500;
end

% Generate parameter
t = 0:(.99*pi/2):N;

% Compute coordinates
x = t .* cos(t);
y = t .* sin(t);

% Plot the curve
plot(x, y, 'k')
axis square
title(sprintf('Pretty Picture with N = %d', N))
xlabel('x = t cos(t)')
ylabel('y = t sin(t)')
end
```

OUTPUT



CODE #2 [help_and_documentation.m]

```
% Call the custom function with 600 points
pretty_picture(600)

% View command line help for 'format' and 'plot'
help format
help plot

% Open the official documentation for 'plot'
doc plot
```

OUTPUT

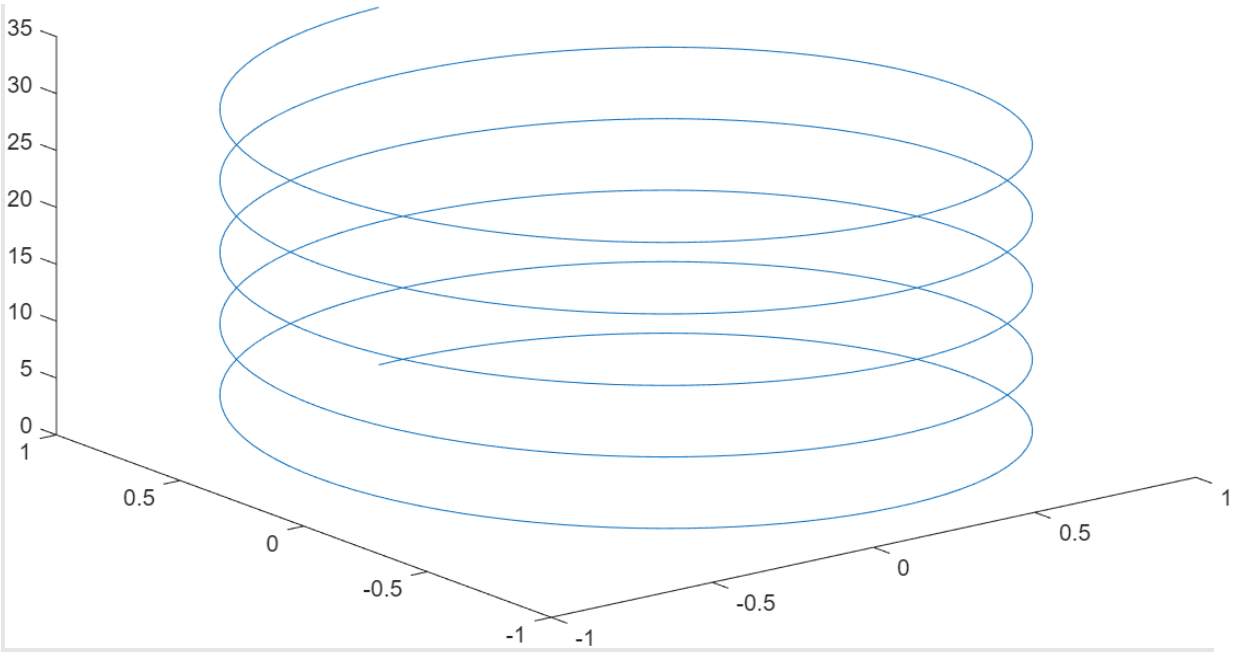
[Show full documentation with link in command line]

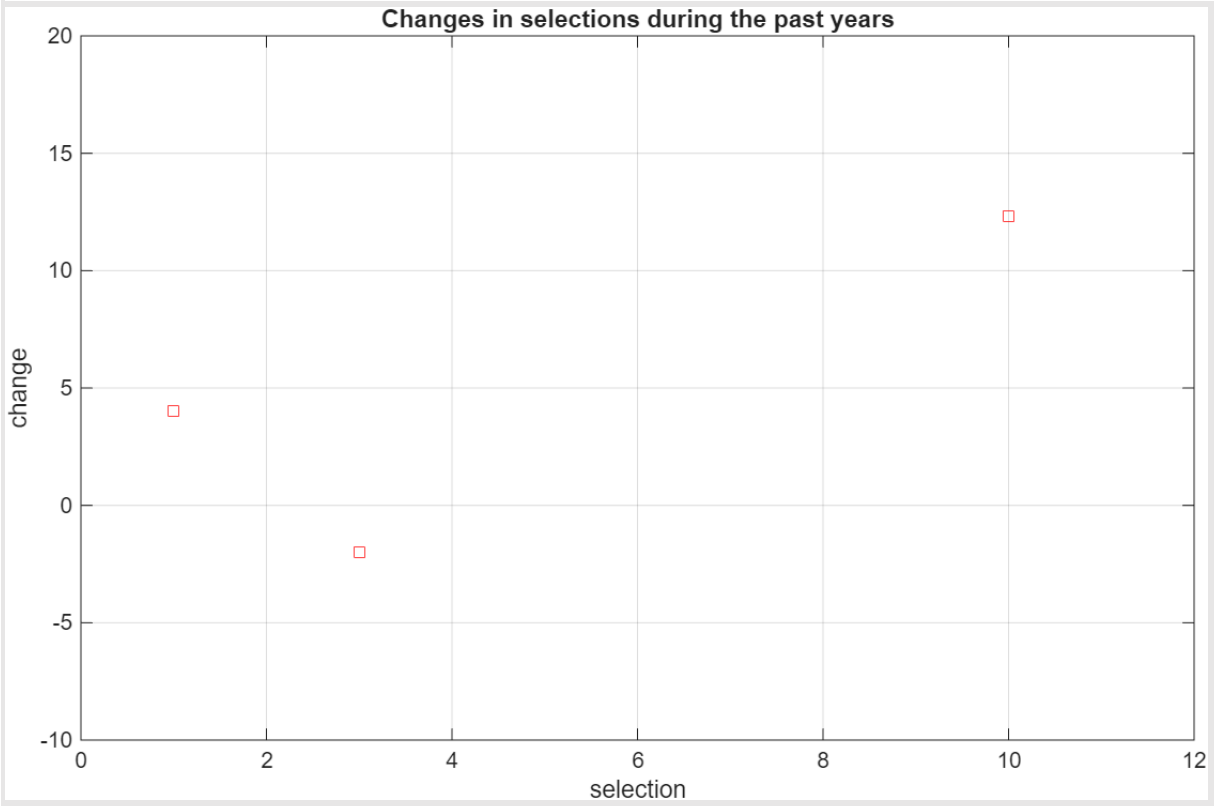
TOPIC – 3: Data Visualization and Plotting

CODE #1 [plot_3d_helix.m]

```
% Create a 3D helix plot
t = 0:pi/50:10*pi;
plot3(sin(t), cos(t), t);
```

OUTPUT


35
30
25
20
15
10
5
0
1
0.5
0
-0.5
-1
-1
-0.5
0
0.5
1


Changes in selections during the past years
change
selection
20
15
10
5
0
-5
-10
0
2
4
6
8
10
12

CODE #2 [plot_2s_and_formatting.m]

```
x_coordinates = [1, 3, 10];
y_coordinates = [4, -2, 12.3];

% Basic 2D plot
plot(x_coordinates, y_coordinates);

% Check how many elements are in the vector
length(x_coordinates)

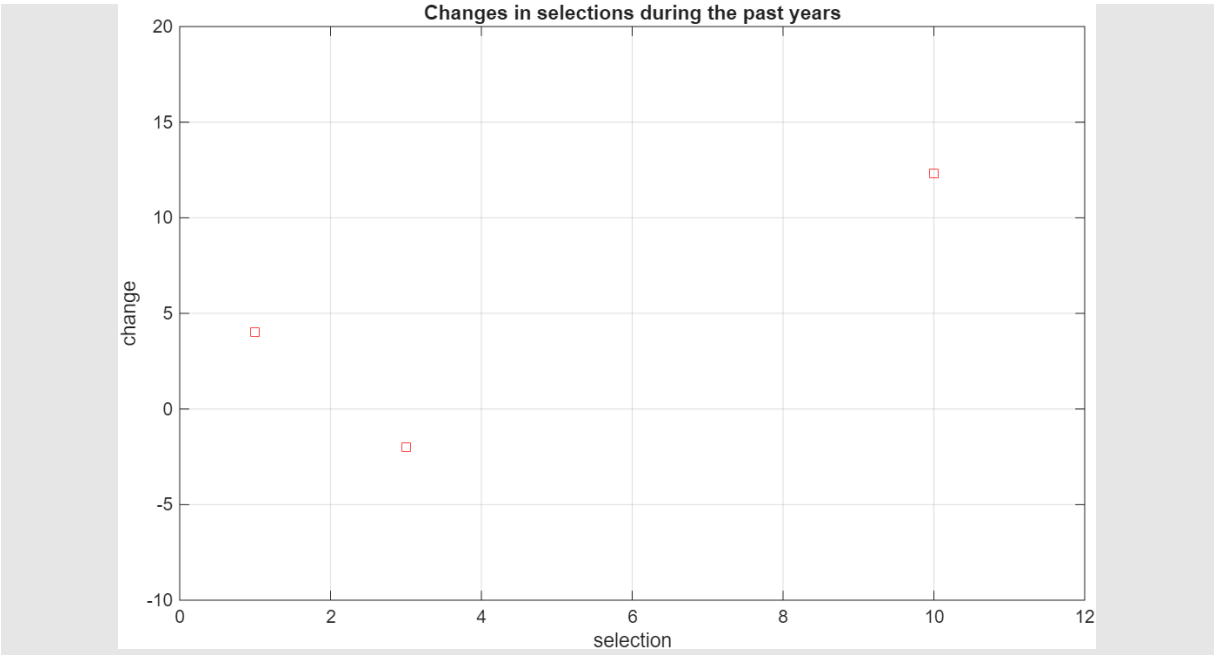
% Plot with just asterisk markers
plot(x_coordinates, y_coordinates, '*');

% Plot with LineSpec: 's' for square marker, 'r' for red line
plot(x_coordinates, y_coordinates, 'sr');

% Formatting the plot
grid on
xlabel('selection')
ylabel('change')
title('Changes in selections during the past years')

% Set manual axis limits: [xmin, xmax, ymin, ymax]
axis([0, 12, -10, 20])
```

OUTPUT



CODE #3 [charts_and_figures.m]

```
x_coordinates = [1, 3, 10];
y_coordinates = [4, -2, 12.3];

% Create a bar chart
bar(x_coordinates, y_coordinates);

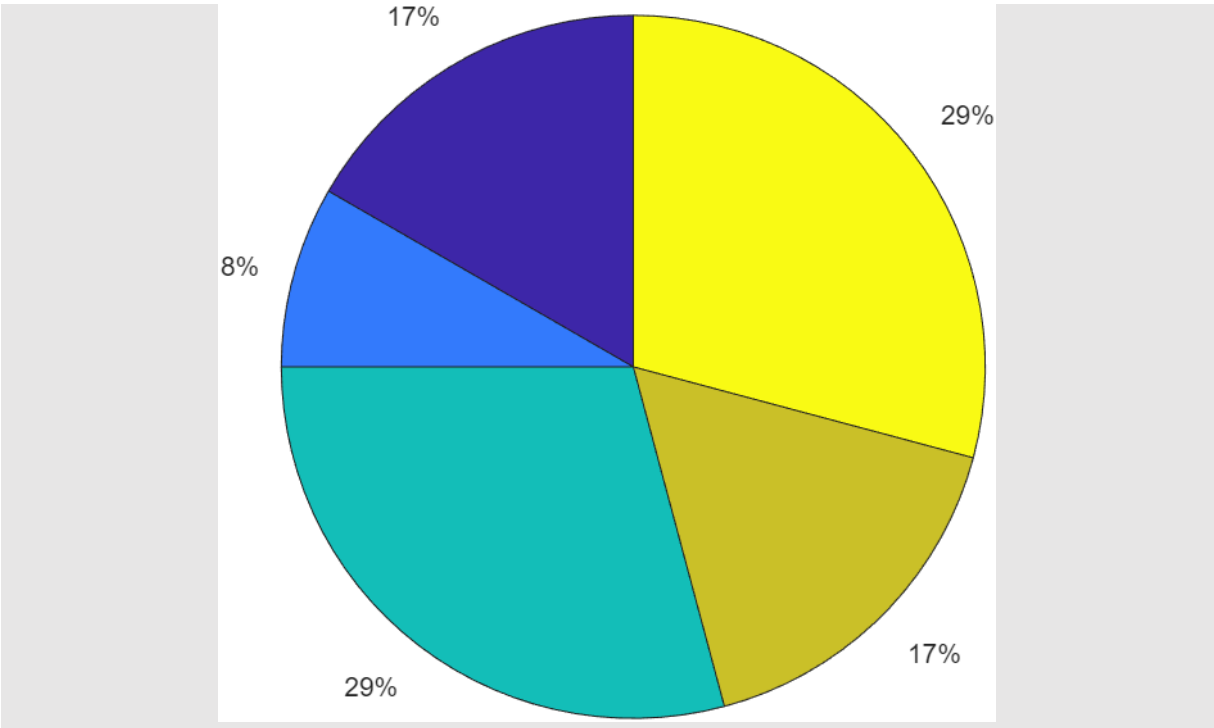
% Open a new blank figure window
figure

% Create a pie chart (spaces or commas work)
pie([4 2 7 4 7])

% Open another figure window and create another pie chart
figure
pie([4, 2, 7, 4, 7])

% Close specific figures by their number
close(1)
close(3)
```

OUTPUT



TOPIC – 4: Image Processing Basics

CODE #1 [read_and_display_image.m]

```
% Read the image file into a variable array
nice_pic = imread('rezwan.jpg');

% Display the image in a figure
image(nice_pic)
```

OUTPUT



TOPIC – 5: Introduction to Matrices and Operators

CODE #1 [basic_math.m]

```
% Built-in math functions
sqrt(2)  % Square root
sin(30)  % Sine (angle in radians)
sind(30) % Sine (angle in degrees)

% Operator vs Function
1 + 3
plus(1, 2)
```

OUTPUT

```
ans =
   3

ans =
   1.4142
ans =
  -0.9880
ans =
   0.5000
ans =
   4
ans =
   3
```

CODE #2 [matrix_creation.m]

```
% Row vectors
xvalues = [1, 2, 10];
row1_voltages = [1 2 3];
x = [1 4 7];

% Column vector (using semicolons)
y = [1; 4; 7];

% 2D Matrices
x_mat = [0 1 -1; 2.5 pi 100];
voltages = [1 2 3; 4 5 6; 7 8 9];

% Create an empty array
empty_arr = [];

% Check sizes
size(5)
size(x)
size(y)
size(empty_arr)
```

OUTPUT

```
ans =
   1   1
ans =
   1   3
ans =
   3   1
ans =
   0   0
```

Name	Value	Size	Class
ans	[0,0]	1×2	double
empty_arr	[ ]	0×0	double
row1_voltages	[1,2,3]	1×3	double
voltages	[1,2,3;4,5,6;7,8,9]	3×3	double
x	[1,4,7]	1×3	double
x_mat	[0,1,-1;2.5000,3.141...	2×3	double
xvalues	[1,2,10]	1×3	double
y	[1;4;7]	3×1	double

CODE #3 [colon_operator.m]

```
% Syntax: start:step:stop
x = 1:3:7;  % Output: 1, 4, 7
x = 1:4:8;  % Output: 1, 5
x = 1:5:10; % Output: 1, 6

% Default step is 1
ints = 1:100;
size(ints)

% Function equivalent of the colon operator
colon(1, 7)

% Odd and even numbers
odd_number = 1:2:100;
size(odd_number)

even_number = 2:2:100;
size(even_number)

% Negative steps
7:-3:1
down_by_10 = 100:-10:-100;
size(down_by_10)
```

OUTPUT

```
ans =
   1   100
ans =
   1   2   3   4   5   6   7
ans =
   1   50
ans =
   1   50
ans =
   7   4   1
ans =
   1   21
```

CODE #4 [matrix_indexing.m]

```
x = [1 2 3 4; 5 6 7 8; 9 10 11 12];

% Read specific elements (row, column)
x(2,3)
alice = x(2,3);
bob = x(3,3);

% Modify specific elements
x(2,3) = 97;
x(2,2)
x(2,2) = 123;
```

OUTPUT

```
ans =
   7
ans =
   6
```

CODE #5 [sub_arrays.m]

```
x = [1 2 3; 4 5 6];

% Extract specific rows and columns
x(2, [1 3])       % 2nd row; 1st and 3rd columns
x([2 1], 2)       % 2nd and 1st rows; 2nd column
x([2,1], [2,3])   % 2nd and 1st rows; 2nd and 3rd columns
x(2, [1 2 3])     % 2nd row; columns 1, 2, and 3
x(2, 1:3)         % Same as above using colon operator
```

OUTPUT

```
ans =
   4   6
ans =
   5
   2
ans =
   5   6
   2   3
ans =
   4   5   6
ans =
   4   5   6
```

CODE #6 [the_end_keyword.m]

```
x = [1 2 3 4; 5 6 7 8; 9 10 11 12];

% 'end' refers to the last index of that dimension
x(end, 2)
x(2, end)
x(end, end)
x([2 end 1 end], 3)
x(1, end-1)
x(1, [end, end-1])

% Expand array by adding a new row at the end
x(end+1, 1) = 17;

% Skip elements using step sizes with 'end'
x(1, 1:2:end)
x(1, end:-1:2)

% Select entire rows or columns using just the colon (:)
x(1:end, 2)
x(:, 2) % Shorthand for "all rows, 2nd column"
x(1, :) % All columns in 1st row
x(:, :) % Entire matrix
```

OUTPUT

```
ans =
  10
ans =
   8
ans =
  12
ans =
   7
  11
   3
  11
ans =
   3
ans =
   4   3
ans =
```

```
    1    3
ans =
    4    3    2
ans =
    2
    6
   10
    0
ans =
    2
    6
   10
    0
ans =
    1    2    3    4
ans =
    1    2    3    4
    5    6    7    8
    9   10   11   12
   17    0    0    0
```

CODE #7 [matrix_arithmetic_concepts.m]

```
% Matrix and Operator Operations[cite: 1]

%% 1. Matrix Transposition & Concatenation[cite: 1]
A = [1 2; 3 4];
B = [5 6; 7 8];

% Transpose a matrix
A_transposed = A'

% Combine matrices horizontally and vertically
combined_horizontal = [A, B]
combined_vertical = [A; B]

%% 2. Array Addition & Subtraction[cite: 1]
% Operands must have the same size
sum_matrix = A + B
diff_matrix = A - B

%% 3. Array Multiplication vs. Matrix Multiplication[cite: 1]
% Element-wise multiplication (operands must have the same shape)[cite: 1]
array_mult = A .* B

% Matrix multiplication (inner dimensions must match)[cite: 1]
matrix_mult = A * B

%% 4. Matrix Division (Solving Linear Systems)[cite: 1]
% Element-wise division
array_div = A ./ B

% Left Division (\): Solves for X in A * X = B[cite: 1]
X_left = A \ B

% Right Division (/): Solves for X in X * B = A[cite: 1]
X_right = A / B

%% 5. Exponential Operations[cite: 1]
% Element-wise power (raises each element individually)[cite: 1]
array_power = A .^ 2

% Matrix power (A * A)[cite: 1]
matrix_power = A ^ 2
```

OUTPUT

```
A_transposed =
   1    3
   2    4
combined_horizontal =
   1    2    5    6
   3    4    7    8
combined_vertical =
   1    2
   3    4
   5    6
   7    8
```